

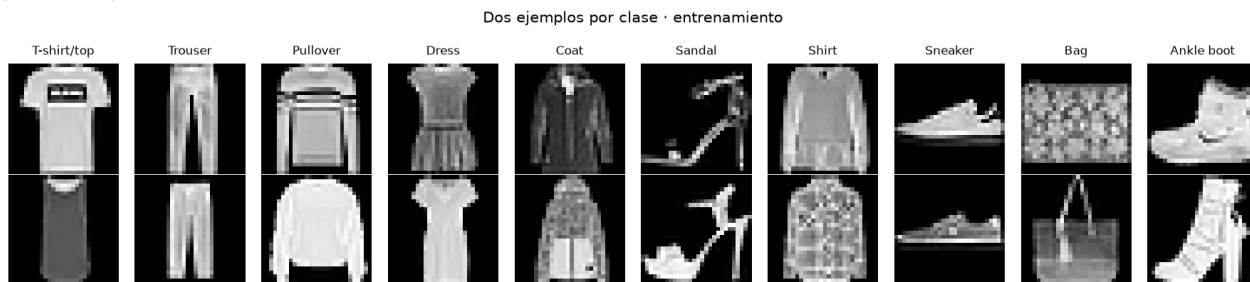
Ejercicio 04 · Clasificador visual con CNN y Model Card

Marlenis Judith Concepción Cuevas · INF-8239-C2 · Unidad II · Docente: Edwin Ramón José Nolasco

Repositorio: <https://github.com/marlenis-concepcion/CienciaDatosUasd2026/tree/main/Ciencia%20de%20DatosII/Unidad%20II/Ejercicio%2004>

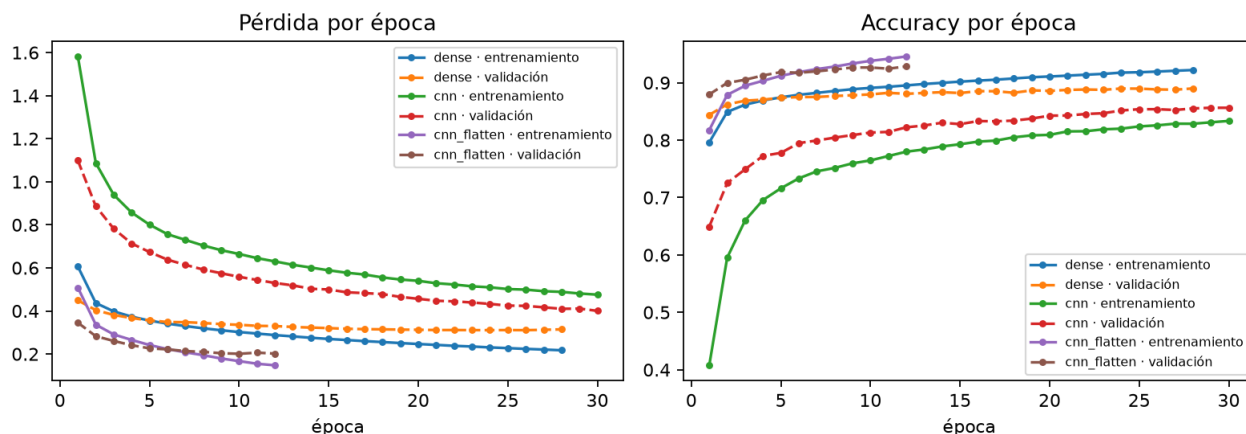
1. Auditoría y partición reproducible

Fashion-MNIST (Xiao, Rasul y Vollgraf, 2017; licencia MIT), cargado con tf.keras. 60,000 imágenes de entrenamiento y 10,000 de prueba de 28×28, balanceadas (6000 y 1000 por clase). La auditoría por hash no encontró duplicados en entrenamiento (0), ni en prueba (0), ni imágenes de entrenamiento repetidas en prueba (0). La validación son 6000 imágenes estratificadas del entrenamiento oficial (semilla 42, índices guardados); la prueba oficial se usó una sola vez, después de fijar la decisión.



2. Línea base y CNN

Se comparan la clase más frecuente, el modelo denso y la CNN pequeña del proyecto base, y una CNN propia que sustituye el promedio global por Flatten + Dense(128). Todos usan Adam, lotes de 128 y parada temprana sobre la pérdida de validación (paciencia 2), con un máximo de 30 épocas. Con las 8 épocas del proyecto base, la CNN pequeña obtuvo F1 0.785 en prueba, por debajo del denso; las curvas muestran que no había convergido.



3. Costo, tamaño y decisión técnica

Modelo	F1 valid.	F1 test	Accuracy test	Parámetros	Épocas	Entrenamientos	Inferencia ms/img	Tamaño KB
Línea base	0.018	0.018	0.100	0	—	—	—	—
Denso (base)	0.888	0.876	0.877	50,890	28	10.9	0.008	619
CNN pequeña (base)	0.854	0.847	0.850	19,466	30	188.2	0.028	262
CNN Flatten	0.926	0.919	0.920	421,642	12	86.8	0.032	4980

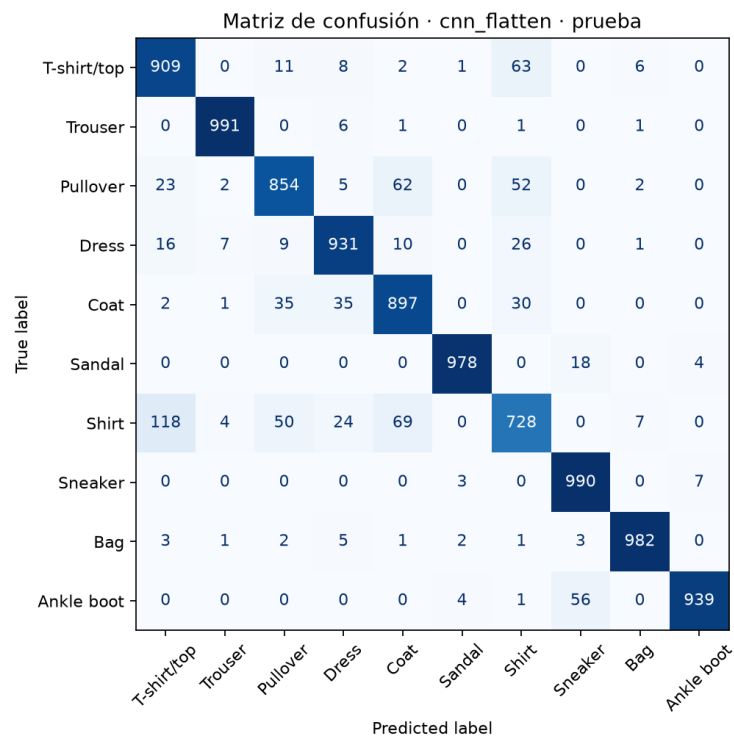
Regla fijada antes de la prueba: menos parámetros entre los modelos a menos de 0.01 del mejor F1 macro de validación. F1 macro de validación máximo 0.9263; dentro de la tolerancia 0.01: cnn_flatten. Se elige el de menos parámetros: cnn_flatten. La CNN Flatten gana +0.04 de F1 macro frente al denso, a cambio de 8 veces más parámetros, 4 veces más inferencia y 5 MB en disco. En CPU (Apple M1 Pro, TensorFlow 2.21) la inferencia sigue por

debajo de 0.1 ms por imagen, por lo que el costo es aceptable.

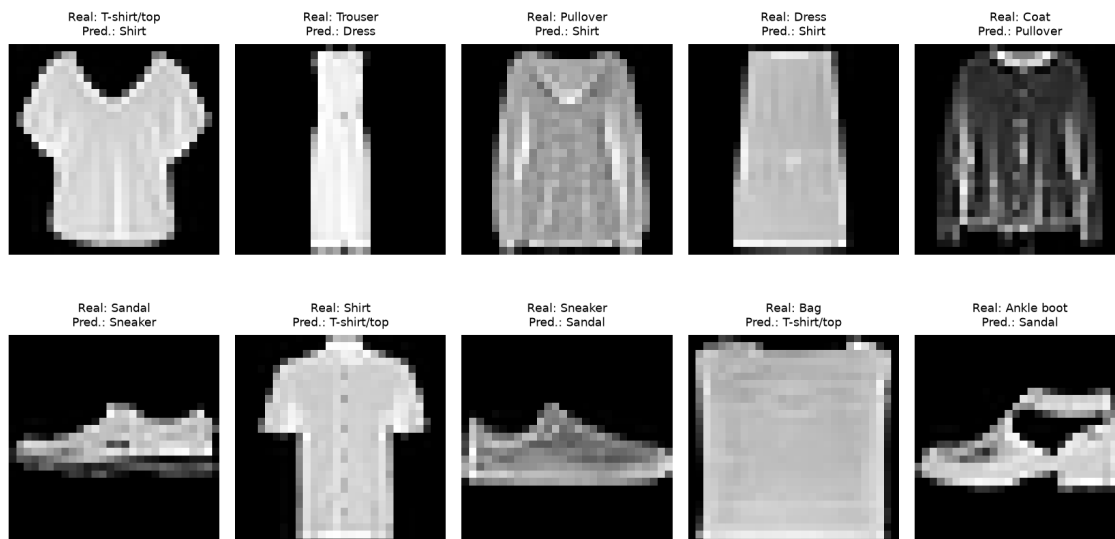
4. Métricas, curvas y errores por clase

Clase	Precisión	Recall	F1	Errores
T-shirt/top	0.849	0.909	0.878	91
Trouser	0.985	0.991	0.988	9
Pullover	0.889	0.854	0.871	146
Dress	0.918	0.931	0.925	69
Coat	0.861	0.897	0.879	103
Sandal	0.990	0.978	0.984	22
Shirt	0.807	0.728	0.766	272
Sneaker	0.928	0.990	0.958	10
Bag	0.983	0.982	0.982	18
Ankle boot	0.988	0.939	0.963	61

Real	Predicha	Errores
Shirt	T-shirt/top	118
Shirt	Coat	69
T-shirt/top	Shirt	63
Pullover	Coat	62
Ankle boot	Sneaker	56



Primer error de prueba por clase real



Shirt es la clase más débil (recall 0.728): se confunde con T-shirt/top, Coat y Pullover, prendas con la misma silueta a 28×28 píxeles. Los errores de calzado (Ankle boot → Sneaker) son el otro grupo relevante. Trouser, Bag, Sandal y Sneaker superan 0.95 de F1.

5. Model Card y reproducibilidad

La Model Card (MODEL_CARD.md) documenta uso previsto, usos fuera de alcance, datos, métricas por clase, costo, límites y monitoreo. Los modelos guardados reproducen las métricas al recargarlos en el cuaderno.

6. Pruebas automatizadas

9 pruebas, todas aprobadas (reports/pruebas.txt): 3 del proyecto base y 6 propias sobre partición, fuga, contrato de los modelos, decisión y conteo de errores. Lista completa en docs/PRUEBAS.md.

#	Prueba	Qué comprueba	Por qué se hizo
1	test normalize adds channel and scales <i>Proyecto base · Auditoría y partición</i>	Las imágenes pasan de 0–255 a 0–1 y reciben el canal (28×28×1).	La red espera ese formato y esa escala; con píxeles sin escalar el entrenamiento es inestable.
2	test validate rejects wrong shape <i>Proyecto base · Auditoría y partición</i>	Falla si una imagen no tiene la forma 28×28×1.	Detiene el proceso antes de entrenar con datos mal cargados en lugar de fallar dentro de Keras con un error poco claro.
3	test cnn output contract <i>Proyecto base · Baseline y CNN</i>	La CNN del proyecto base devuelve 10 probabilidades por imagen que suman 1.	Confirma que la salida es una distribución válida sobre las 10 clases; sin eso las métricas no tienen sentido.
4	test other models output probabilities[build dense] <i>Nueva · Baseline y CNN</i>	El modelo denso cumple el mismo contrato: 10 probabilidades que suman 1.	Los tres modelos se comparan con las mismas métricas, así que deben producir el mismo tipo de salida.
5	test other models output probabilities[build cnn flatten] <i>Nueva · Baseline y CNN</i>	La CNN Flatten (la elegida) cumple el mismo contrato.	Es el modelo que se entrega; se verifica igual que el del profesor.
6	test validation split is disjoint stratified and reproducible <i>Nueva · Auditoría y partición</i>	Entrenamiento y validación no comparten imágenes, suman el total, cada clase tiene la misma cantidad en validación y con la semilla 42 la partición es idéntica al repetirla.	El proyecto base tomaba las últimas 6000 imágenes sin estratificar; la prueba garantiza una validación equilibrada y reproducible, sin tocar la prueba oficial.
7	test audit detects train images repeated in test <i>Nueva · Auditoría y partición</i>	La auditoría por hash detecta una imagen de entrenamiento copiada en prueba.	En Fashion-MNIST se encontraron 0 copias; la prueba demuestra que la auditoría sí las habría encontrado, así que ese 0 es confiable.
8	test choose prefers cheaper model within tolerance <i>Nueva · Costo, tamaño y decisión</i>	Si dos modelos están a menos de 0.01 de F1, se elige el de menos parámetros; si la diferencia es mayor, se elige el mejor.	Fija la regla de decisión técnica antes de ver la prueba: solo se paga más costo si la mejora es real.

#	Prueba	Qué comprueba	Por qué se hizo
9	test per class and confusions count errors <i>Nueva · Métricas y errores por clase</i>	Los errores por clase y las confusiones principales (por ejemplo Shirt → T-shirt/top) se cuentan correctamente en un caso pequeño calculado a mano.	Las tablas de errores por clase del PDF y la Model Card salen de estas funciones; la prueba confirma que los conteos son correctos.

7. Uso de IA

Utilicé Claude Code (Claude Opus 5.5), Codex (OpenAI) y DeepSeek como apoyo. Con Claude Code adapté el proyecto base, propuse la CNN Flatten y escribí las pruebas, el cuaderno y la redacción. Verifiqué la corrida de 8 épocas, la reproducción de métricas desde los modelos guardados, la partición y las pruebas; se corrigió la regla de decisión y la partición de validación. Detalle en docs/DECLARACION_IA.md.

Herramienta	Modelos	Uso en esta práctica	Para qué sirve cada modelo	Control
Claude Code (Anthropic)	Claude Opus 5.5 · Claude Sonnet 5 · Claude Haiku 4.5	Agente principal (Opus 5.5): código, pruebas, cuaderno, documentación, ejecución y PDF	Opus 5.5: tareas largas de varios pasos · Sonnet 5: programación cotidiana · Haiku 4.5: tareas rápidas y baratas	Todo se verificó con pruebas y reportes
Codex (OpenAI)	GPT-5-Codex · codex-mini	Apoyo complementario	GPT-5-Codex: proponer y revisar cambios de código en el repositorio · codex-mini: respuestas rápidas en terminal	Se acepta solo si pasa las pruebas
DeepSeek	DeepSeek-V3 · DeepSeek-R1 · DeepSeek-Coder-V2	Apoyo complementario	V3: explicar conceptos y revisar redacción · R1: razonamiento paso a paso sobre métricas · Coder-V2: generar código	Referencias verificadas en la fuente

Reparto de responsabilidades

Responsabilidad	Quién
Valores del análisis (daño prioritario, tolerancias, compuertas, decisiones de uso)	La autora
Elección de datos y verificación de licencias y referencias	La autora, con apoyo de las herramientas
Borradores de código, pruebas y redacción	Herramientas de IA
Ejecución de pruebas y comprobación de cada cifra contra reports/	La autora, con Claude Code
Defensa oral y respuesta a preguntas técnicas	La autora